Having named our image, we're now able to start our Rails server using the image name, like so:

```
$ docker run -p 3000:3000 railsapp \
    bin/rails s -b 0.0.0.0
```

Ahh. Much better.

Running a specific version of an image is exactly as you'd expect: using the same colon notation as used previously. For example, to explicitly use version 1.0 of our image, we'd run:

```
$ docker run -p 3000:3000 railsapp:1.0 \
    bin/rails s -b 0.0.0.0
```

A Default Command

Currently, every time we want to start a Rails server in a container, we have to explicitly specify the command bin/rails s -b 0.0.0.0 as part of our docker run command:

```
$ docker run -p 3000:3000 railsapp \
    bin/rails s -b 0.0.0.0
```

This is a shame because the *main purpose* of our custom image is to start a Rails server. It would be better if we could embed the knowledge of how to start the Rails server in the image itself.

We can do this by adding a new instruction to our Dockerfile. The CMD instruction, pronounced "command," specifies the *default command* to run when a container is started from the image. Let's use this in our Dockerfile to start the Rails server by default:

```
FROM ruby:2.6

RUN apt-get update -yqq
RUN apt-get install -yqq --no-install-recommends nodejs

COPY . /usr/src/app/

WORKDIR /usr/src/app
RUN bundle install

➤ CMD ["bin/rails", "s", "-b", "0.0.0.0"]
```

Looking at this new line, you may notice the weird array notation used to specify the command. This form—known as the *Exec* form—is needed so that our Rails server is started as the first process in the container (PID 1) and correctly receives Unix signals such as the signal to terminate. It's the recommended form, and most commonly used.